

```
2516 /home/user/audit/live_ml_bot.py
#!/usr/bin/env python3
"""
...
Overlays pre-trained champion inference and computes rolling technical indicators
to run 5 diverse quantitative strategies simultaneously:
1. ML Mean Reversal (LightGBM/RF)
2. CVD Momentum (Trend Following)
```

Exit Code: 0

Using Tool |  Bash Command `grep -n "def \\class \\_rebuild_queues\\|candle_lock\\|active_trades_lock..."` View

```
61: def ist_str():
74: def parse_gdrive_ts(ts_str):
88:     except Exception as e:
...
2383:         trades_snap = dict(ex.active_trades)
2400:         except Exception:
2462:     def start(self):
2507:     except KeyboardInterrupt:
```

Exit Code: 0

Now let me examine the critical sections in depth - the data pipeline, thread safety, strategy gating, and risk management areas.

Using Tool |  Read File `/home/user/audit/live_ml_bot.py` View

```
100     2. CVD Momentum (Trend Following)
101     3. CVD Channel Breakout (Breakout)
102     4. VWAP Deviation Bounce (Bounce)
...
316         rs = avg_gain / avg_loss
317         rsi_list[period] = 100.0 - (100.0 / (1.0 + rs))
318
319     # Wilder's smoothing for subsequent values
```

Using Tool |  Read File `/home/user/audit/live_ml_bot.py` View

```
320     for i in range(period + 1, len(closes)):
321         avg_gain = (avg_gain * (period - 1) + gains[i - 1]) / period
322         avg_loss = (avg_loss * (period - 1) + losses[i - 1]) / period
...
516     except Exception as e:
517         _log.error("[ML] [%s] Error loading G Drive seed file: %s", self.symbol, e)
518
519     # 2. Next, load and merge from local cache (contains previously hand-scraped candles)
```

Using Tool |  Read File `/home/user/audit/live_ml_bot.py` View

```
520     local_file = os.path.join(PROJ_DIR, f"seed_data_{self.symbol}.csv")
521     if os.path.exists(local_file):
522         try:
...
796
797         # cvd_flip (1-period CVD velocity)
798         cvd_flip = cvd_arr[-1] - cvd_arr[-2] if len(cvd_arr) >= 2 else 0.0
799
```

Using Tool |  Read File `/home/user/audit/live_ml_bot.py` View

```
800         # rsi
801         rsi = rsi_arr[-1]
802
...
1136     self.pending_trades = set() # Prevent async double execution
1137     self.last_bid = 0.0
1138     self.last_ask = 0.0
1139     self._trade_lock = threading.Lock() # Protects active_trades + pending_trades from
concurrent threads
```

Using Tool |  Read File `/home/user/audit/live_ml_bot.py` View

```
1140     self.engine_ref = None # Link to MLMacroMicroEngine for live ATR access
1141
1142     def _get_current_atr(self):
...
1496         max_allowed_nominal = max(0.0, max_allowed_nominal - current_nominal)
1497         max_qty = max_allowed_nominal / entry
1498
1499         if volume > max_qty:
```

Using Tool | Read File /home/user/audit/live_ml_bot.py

View

```
1499         if volume > max_qty:
1500             _log.info("[ML-Executor] [%s] [%s] Volume %.2f lot capped at %.2f lot to enforce
global 5.0x leve...
1501                 self.symbol, strategy_name, volume, max_qty, capital)
...
1845             _log.info("[ML-Executor] [%s] [%s] S-7 Partial Close Tier 2: Closed
%.2f lots (50%% o...
1846                 self.symbol, name, close_vol, price_for_pnl)
1847
1848             # LONG POSITION trailing stops logic
```

Using Tool | Read File /home/user/audit/live_ml_bot.py

View

```
1848             # LONG POSITION trailing stops logic
1849             if side == "LONG":
1850                 lock_price = entry + risk * rr * lock_pct
...
2194                 # Pre-reserve slot (Bug C-4 / M-5)
2195                 ex.pending_trades.add(strat_name)
2196                 active_count += 1
2197                 if sig.get("fire_long"):
```

Using Tool | Read File /home/user/audit/live_ml_bot.py

View

```
2197                 if sig.get("fire_long"):
2198                     has_long = True
2199                     net_exposure += 1
...
2513
2514
2515 if __name__ == "__main__":
2516     MLLiveOrderFlowBot().start()
```

Now I have full visibility into the code. Let me compile the deep adversarial Red-Team audit. Let me also probe one more area – the funding-rate seed logic – before finalizing:

Using Tool | Bash Command grep -n "funding_rate\\|fr_extreme\\|fr_arr\\|fr_current" /home/user/audit/l...

View

```
250:         "funding_rate": 0.0,
355:         "funding_rate": c_snap.get("funding_rate", 0.0),
510:         "funding_rate": _safe_float(row.get("Agg. Funding Rate"), 0.0),
...
1087:         s["fr_extreme_long"] = fr_extreme_long
1088:         s["fr_extreme_short"] = fr_extreme_short
2183:             if sig.get("fr_extreme_long") and sig.get("fire_long"):
2186:                 elif sig.get("fr_extreme_short") and sig.get("fire_short"):
```

Exit Code: 0

I have enough context. Now let me deliver the comprehensive Deep Bug Analysis & Action Report.

Deep Bug Analysis & Action Report — live_ml_bot.py v5.0

After an adversarial red-team audit of all 2,516 lines across the data pipeline, threading, gating logic, risk management, and resiliency vectors, I found **18 confirmed defects** ranging from **CRITICAL (capital-loss-causing)** down to **LOW (cosmetic)**. They are ordered by severity.

CRITICAL — Capital-at-risk bugs

BUG C-1 · `manage()` partial-close PnL is computed BEFORE the broker actually closes (DRY mistakenly assumes always-success, LIVE double-counts)

Location: `MLMT5Executor.manage()` lines 1796–1846 **Impact:** In LIVE mode, if `_close_position()` returns `True` for a *partial* close, the parent position's `t["volume"]` is decremented locally — but if the broker actually executed `ORDER_FILLING_IOC` and only *partially* filled the partial (TRADE_RETCODE_DONE_PARTIAL, retcode 10009), the local book and broker book desync. Then Tier-2 fires on the WRONG volume baseline and over-closes. In DRY_RUN, `partial_ok=True` is hardcoded, masking the bug entirely during paper trading. **Root Cause:** `_close_position()` returns `True` on both `DONE` and `DONE_PARTIAL` (line 1339) but does not return the *actually filled* close volume. The caller

assumes the full `close_vol` was closed. **Fix:**

```
# In _close_position(), return the filled volume instead of bool:
def _close_position(self, position, side, bid, ask, strategy_name, volume_to_close=None):
    ...
    for attempt in range(CLOSE_RETRIES):
        ...
        with _MT5_LOCK:
            res = mt5.order_send(req)
            _CLOSE_SUCCESS = {mt5.TRADE_RETCODE_DONE, getattr(mt5, "TRADE_RETCODE_DONE_PARTIAL",
10009)}
            if res and res.retcode in _CLOSE_SUCCESS:
                return float(getattr(res, "volume", vol)) # return real filled volume
            ...
    return 0.0

# In manage() Tier 1 (and Tier 2 identically), replace:
partial_ok = self._close_position(pos, side, bid, ask, name, volume_to_close=close_vol)
# with:
filled_close_vol = self._close_position(pos, side, bid, ask, name,
volume_to_close=close_vol)
if filled_close_vol >= t_min_v:
    t["volume"] = round(t["volume"] - filled_close_vol, t_decimals)
    if DRY_RUN_MODE:
        _cs = self._get_contract_size()
        _partial_pnl = ((price_for_pnl - entry) if side == "LONG" else (entry -
price_for_pnl)) * filled_close_vol * _cs
        self.session_pnl += _partial_pnl
    t["virtual_sl"] = entry
    new_vsl = entry
    t["partial_exits_done"] = 1
```

 **BUG C-2 · fixed_tp override silently flips the TP through the entry price for VWAP_Bounce shorts when signal_tp > fixed_tp**

Location: `open_trade()` lines 1421–1429 **Impact:** For `VWAP_Bounce` (`rr=2.0`, no trail), the engine carefully computes a mean-reversion TP at the VWAP midline (lines 1037–1043). But `open_trade()` then overrides:

```
tp_price = max(signal_tp, fixed_tp) if (signal_tp < entry) else fixed_tp
```

For a **SHORT**, if the broker's MT5 entry (ask) is slightly **above** the signal's `coinglass_entry`, the VWAP midline (`signal_tp`) may end up **above** entry. The condition `signal_tp < entry` evaluates `False`, and it silently falls back to `fixed_tp = entry - risk*2.0`, throwing away the carefully-computed reversion target. Worse, **risk here uses MT5 entry** but `sl_price` was computed from `coinglass_entry`, so risk distance can become microscopic or even negative on price gaps → triggers `min_risk_dist` floor and produces an illogical TP. **Root Cause:** The "preserve signal TP" logic doesn't recompute `signal_tp` against the actual MT5 fill, and silently discards a strategy-specific TP without logging. **Fix:**

```
signal_tp = tp_price
fixed_tp = (entry + risk * s_config["rr"]) if side == "LONG" else (entry - risk *
s_config["rr"])

# Re-anchor the signal TP to the actual MT5 entry to preserve the strategy's R:R intent
if coinglass_entry and coinglass_entry > 0:
    tp_offset = signal_tp - coinglass_entry # preserve direction/magnitude
    rebased_signal_tp = entry + tp_offset
else:
    rebased_signal_tp = signal_tp

if side == "LONG":
    if rebased_signal_tp > entry:
        tp_price = min(rebased_signal_tp, fixed_tp)
    else:
        _log.warning("[ML-Executor] [%s] [%s] signal TP %.2f is on wrong side of entry %.2f;
using fixed R:R TP %.2f",
                    self.symbol, strategy_name, rebased_signal_tp, entry, fixed_tp)
        tp_price = fixed_tp
else:
    if rebased_signal_tp < entry:
        tp_price = max(rebased_signal_tp, fixed_tp)
    else:
        _log.warning("[ML-Executor] [%s] [%s] signal TP %.2f is on wrong side of entry %.2f;
using fixed R:R TP %.2f",
                    self.symbol, strategy_name, rebased_signal_tp, entry, fixed_tp)
        tp_price = fixed_tp
```

🐛 **BUG C-3 · pending_trades slot leak when multiple symbols share the executor logic AND the limit check is per-executor while the 7-cap is global in spirit**

Location: `open_trade()` lines 1358–1367, `_on_update()` lines 2140–2207 **Impact:** Each symbol gets its OWN `MLMT5Executor` instance (`self.executors[symbol] = MLMT5Executor(symbol)`, line 2078 / 2498), and each instance has its own `_trade_lock` and its own `active_trades/pending_trades`. The 7-trade portfolio cap is enforced **per-symbol**, NOT portfolio-wide. With 3 symbols connected, the bot can open up to **21 concurrent trades** while believing it respects a "7 active/pending" cap. The 5x leverage cap (line 1485) is also per-symbol because `acc_info.equity` is read for the global account, but `current_nominal` only sums *this executor's* trades. **Root Cause:** No portfolio-level aggregation. Each symbol's executor is fully independent. **Fix:** Introduce a class-level (or bot-level) atomic registry:



```
# Add at module level
_PORTFOLIO_LOCK = threading.Lock()
_PORTFOLIO_ACTIVE = {} # {(symbol, strat): {"side":..., "nominal":...}}

# Replace per-executor check in open_trade():
with _PORTFOLIO_LOCK:
    total_open = sum(1 for _ in _PORTFOLIO_ACTIVE)
    if total_open >= 7:
        _log.warning("[ML-Executor] [%s] [%s] Portfolio 7-trade cap reached (%d open across all symbols).",
                    self.symbol, strategy_name, total_open)
        return False
    key = (self.symbol, strategy_name)
    if key in _PORTFOLIO_ACTIVE:
        return False
    _PORTFOLIO_ACTIVE[key] = {"side": side, "nominal": 0.0, "pending": True}

# After successful open:
with _PORTFOLIO_LOCK:
    _PORTFOLIO_ACTIVE[key] = {"side": side, "nominal": filled_vol * real_fill, "pending": False}

# On close (in manage()):
with _PORTFOLIO_LOCK:
    _PORTFOLIO_ACTIVE.pop((self.symbol, name), None)

# Similarly, compute current_nominal globally for leverage cap:
with _PORTFOLIO_LOCK:
    current_nominal = sum(v["nominal"] for v in _PORTFOLIO_ACTIVE.values())
```

🐛 **BUG C-4 · CVD_Momentum short condition uses `abs(oi_mom)` — fires aggressively on rising OI sell-offs where OI should confirm the trend, not be ambiguous**

Location: `_evaluate_m1()` line 986

```
mom_short = (mom5 <= -0.0050 and cvd_flip < 0 and abs(oi_mom) >= 0.010)
```

Impact: The comment claims "falling OI during sell-offs is also institutional activity" — but `abs(oi_mom) >= 0.010` also fires when OI is **rising** during the sell-off, which is the opposite signal: bullish absorbers loading into the dip. The long leg correctly requires `oi_mom >= 0.010` (rising OI confirms breakout). The short leg's symmetric requirement should be `oi_mom <= -0.010` (falling OI = capitulation/exhaustion exit). Using `abs()` will fire on **both** longs and shorts confirming OI, leading to whipsaw losses. **Root Cause:** Misguided "fix" for the H-2 bug. The symmetric correct condition is `oi_mom <= -0.010`. **Fix:**

```
mom_long = (mom5 >= 0.0050 and cvd_flip > 0 and oi_mom >= 0.010)
mom_short = (mom5 <= -0.0050 and cvd_flip < 0 and oi_mom <= -0.010)
```

🐛 **BUG C-5 · CVD baseline alignment uses `latest_past_cvd` which is the live-injected candle, creating a self-referencing offset that gets computed on every restart**

Location: `update()` lines 644–660 **Impact:** The condition `getattr(self, "cvd_baseline_offset", None) is None` is `True` only on first live tick after startup. But: (a) `past_ts = [t for t in sorted(...) if t < current_ts_sec]` is computed AFTER `self.all_candles_dict[current_ts_sec] = candle` ran earlier in the seeders. If the freshly received live `current_ts_sec` already exists in `all_candles_dict` from Binance API seed (likely, since 5m bars are pre-seeded), then `latest_past_cvd` is the CVD of the **previous 5m candle from Binance** — which is computed as a running sum from the start of the 1500-candle window, NOT the cumulative Coinglass baseline. The offset will be huge (often 10M+ on BTC), then `cvd_baseline_offset` is **persisted to disk** in `cvd_offset*.json` (line 658). On next restart, the offset is **re-loaded** (line 170), and the condition `cvd_baseline_offset is None` is `False`, so the bot skips the "live CVD" logic and uses the stale, off-baseline offset. The "fix" is to ensure the baseline is reset or aligned correctly on restart.

so it's never recomputed — but the Binance seed runs fresh, producing different CVD numbers. The offset becomes stale immediately. **Root Cause:** Persisting an offset that was computed against one specific seed snapshot, then reusing it across restarts where the seed has been re-run from a different CVD origin. **Fix:** Either (a) never persist (recompute fresh every session), or (b) clear the offset before re-seeding:

```
# At top of __init__, BEFORE seeding:
self.cvd_baseline_offset = None
# Delete stale offset file at startup — must always be recomputed against fresh seeds
if os.path.exists(self.offset_file):
    try:
        os.remove(self.offset_file)
        _log.info("[ML] [%s] Cleared stale CVD offset file (will recompute on first live
tick).", self.symbol)
    except Exception as e:
        _log.warning("[ML] [%s] Could not clear CVD offset file: %s", self.symbol, e)

# Also remove lines 167-173 that load the persisted offset.
```

Additionally, in `update()` only apply the offset to **non-Coinglass** candles to preserve the integrity of G-Drive seed CVDs:

```
for t in past_ts:
    cand = self.all_candles_dict[t]
    if "cvd" in cand and not cand.get("is_coinglass", False):
        cand["cvd"] += self.cvd_baseline_offset
```

■ HIGH — Logic/Data-Correctness bugs

🐛 **BUG H-1 · `_rebuild_queues()` recomputes CVD/RSI from sampled-list deltas, overwriting the carefully-seeded Coinglass CVD on every rebuild — defeating the H-4 fix**

Location: `_rebuild_queues()` lines 447–465 **Impact:** The Binance API merge at `seed_from_binance_api()` carefully preserves Coinglass CVD/RSI for `is_coinglass=True` candles (lines 277–290). But `_rebuild_queues()` unconditionally recomputes a running CVD by summing deltas and overwrites `c["cvd"]` for every sampled candle (line 455). Since `sampled_list` references the SAME dict objects in `all_candles_dict` (line 396:

`sampled_list.append(self.all_candles_dict[target_ts]))`, this mutates the *master dict*. After one rebuild, G-Drive CVDs are forever lost. The CVD baseline now becomes whatever Binance delta sum produces (starting from 0 at the beginning of the 25-min-sampled window), which is a totally different scale than the live Coinglass CVD. Then the CVD baseline alignment in `update()` fires with a nonsensical offset.

Root Cause: Naive in-place mutation of shared dict objects. The fix was intended to handle the case where Binance-seeded candles have a frozen CVD, but it now over-corrects. **Fix:** Recalculate CVD/RSI only for the *deque view* without mutating the master dict:

```
if len(sampled_list) > 0:
    # Make local copies so we don't mutate all_candles_dict
    sampled_list = [c.copy() for c in sampled_list]
    sampled_list.reverse()

    # Forward-fill OI as before
    last_real_oi = 0.0
    for c in sampled_list:
        oi_val = c.get("open_interest", 0.0)
        if oi_val > 100:
            last_real_oi = oi_val
        elif last_real_oi > 100:
            c["open_interest"] = last_real_oi

    # Recompute CVD only if the sampled window has NO Coinglass-trusted candles
    # (i.e. cold-start where all candles came from Binance API)
    has_coinglass = any(c.get("is_coinglass", False) for c in sampled_list)
    if not has_coinglass:
        running_cvd = 0.0
        for c in sampled_list:
            running_cvd += c.get("delta", 0.0)
            c["cvd"] = running_cvd
        closes_for_rsi = [c["close"] for c in sampled_list]
        rsi_values = self.calculate_rsi_list(closes_for_rsi, period=14)
        for i, rsi_val in enumerate(rsi_values):
            sampled_list[i]["rsi"] = rsi_val
    # else: keep Coinglass CVD/RSI exactly as seeded
```

🐛 **BUG H-2 · `_rebuild_queues()` reads `all_candles_dict` under lock but then releases the lock and reads same dict objects unsafely during CVD/RSI recompute & OI forward-fill**

Location: `_rebuild_queues()` lines 367–471 **Impact:** The function snapshots `sorted_keys` and references inside the first with `self.candle_lock`: block. After the block exits at line 406, the references in `sampled_list` point to the same dict objects in `self.all_candles_dict`. Mutations to `c["cvd"]`, `c["open_interest"]`, `c["rsi"]` (lines 444, 455, 465) happen **outside the lock**. Meanwhile, the WS thread in `update()` may be writing to `all_candles_dict[current_ts_sec]` (line 683) and reading the same dict via `super().update()`. This is a classic data-race: reads return torn/partial dicts. **Root Cause:** Lock released too early to do in-place mutation safely. **Fix:** Either (a) hold the lock during the full mutation (preferred — these are cheap loops), or (b) snapshot copies inside the lock and never mutate the originals. Combine with H-1 fix above:

```
def _rebuild_queues(self):
    with self.candle_lock:
        # ... (existing eviction + sorted_key ... ) ...
        # Build SHALLOW COPIES inside the lock:
        sampled_list = []
        for step in range(120):
            target_ts = anchor_ts - step * 1500
            if target_ts in self.all_candles_dict:
                sampled_list.append(self.all_candles_dict[target_ts].copy()) # copy!
            else:
                found = False
                for drift in [100, -100]:
                    if (target_ts + drift) in self.all_candles_dict:
                        sampled_list.append(self.all_candles_dict[target_ts + drift].copy())
                        found = True
                        break
                if not found:
                    break
        # Now safe to mutate sampled_list outside the lock – all dicts are private copies
        ...
```

BUG H-3 · `seed_from_binance_api()` is called from inside `_rebuild_queues()` while `candle_lock` is held — **deadlock risk** if requests' connection-pool callback re-enters

Location: `_rebuild_queues()` line 411 within the `if len(sampled_list) < 20`: self-healing block
Impact: Line 411 calls `self.seed_from_binance_api()` after releasing the `candle_lock` (correct), but then re-acquires `self.candle_lock` at line 413. If the WS thread is *waiting* to write a live candle while this self-healing path runs (which performs a multi-second HTTP request), live ticks pile up. Worse, `seed_from_binance_api` itself populates `self.all_candles_dict` (line 274) **without acquiring `candle_lock`** — concurrent writes from the WS thread will collide → `RuntimeError: dictionary changed size during iteration` or torn values. **Root Cause:** `seed_from_binance_api()` writes to `all_candles_dict` without lock protection. Acceptable at `init` (single-threaded) but unsafe when called mid-flight from `_rebuild_queues()`. **Fix:**

```
def seed_from_binance_api(self):
    try:
        # ... HTTP fetch and parse outside the lock ...
        # Then merge under the lock:
        with self.candle_lock:
            added_count = 0
            for c in parsed_candles:
                ts = c["timestamp"]
                if ts not in self.all_candles_dict:
                    self.all_candles_dict[ts] = c
                    added_count += 1
                else:
                    existing = self.all_candles_dict[ts]
                    existing.update({"open": c["open"], "high": c["high"], "low": c["low"],
                                    "close": c["close"], "volume": c["volume"], "delta":
c["delta"]})
                    if not existing.get("is_coinglass", False):
                        existing["cvd"] = c["cvd"]
                        existing["rsi"] = c["rsi"]
            _log.info("[ML] [%s] Merged %d new candles from Binance API ...", self.symbol,
added_count, ...)
        except Exception as e:
            _log.error("[ML] [%s] Error seeding from Binance API: %s", self.symbol, e)
```

BUG H-4 · `_evaluate_ml()` extracts arrays from `self.sampled_candles` deque **without holding the lock** — concurrent rebuild can corrupt `np.array(...)` reads

Location: `_evaluate_ml()` lines 782–791 **Impact:** `self.sampled_candles` is a deque(`maxlen=150`). Although `_rebuild_queues()` writes to it under lock (line 468–471), `_evaluate_ml()` is called from inside `_rebuild_queues()` at line 475 — so on the first call the lock is implicitly held *only because the*

same thread acquired it. But `_evaluate_m1()` does NOT itself acquire the lock, and `self.sampled_candles[-1]` accessed at line 475 is fine while lock is held. Critically, after `_rebuild_queues` returns the lock, any other thread could subsequently call `_evaluate_m1` indirectly via `_panel.*` UI code reading `last_features`, but more dangerously: line 691 calls `self._rebuild_queues()` from a daemon thread `(threading.Thread(target=self.save_candles_to_cache,...))` at the same time as the main WS update path can re-enter through line 736. So two threads can be inside `_rebuild_queues` → `_evaluate_m1` concurrently, both iterating `self.sampled_candles` to build numpy arrays. Deques are not thread-safe for iteration during concurrent mutation. **Root Cause:** No lock around deque-to-array conversion. **Fix:**

```
def _evaluate_m1(self, last_closed):
    ...
    if len(self.sampled_candles) < 20:
        ...
        return
    try:
        # Snapshot deque under lock
        with self.candle_lock:
            sampled_snap = list(self.sampled_candles)
            if len(sampled_snap) < 20:
                return
            close_arr = np.array([c["close"] for c in sampled_snap])
            high_arr = np.array([c["high"] for c in sampled_snap])
            low_arr = np.array([c["low"] for c in sampled_snap])
            cvd_arr = np.array([c["cvd"] for c in sampled_snap])
            rsi_arr = np.array([c["rsi"] for c in sampled_snap])
            # ... rest unchanged, but reference sampled_snap instead of self.sampled_candles
```

🐛 **BUG H-5 · Liq_Exhaustion thresholds use absolute \$10,000 floor but liq_long/liq_short are in quote-currency from Coinglass** — for low-cap symbols (e.g., XRPUSD) liquidations rarely exceed \$10K per candle, permanently blocking the strategy

Location: `_evaluate_m1()` lines 1057–1068 **Impact:** `LIQ_WARMUP_THRESHOLD = 10000.0` is hardcoded. For BTC this is reasonable, but XRP/SOL/lower-cap futures liquidations per 5m candle are often \$1K–\$5K on the rolling mean. `mean_liq_long >= 10000.0` will be False ~95% of the time on XRP → `liq_l_spike` always False → `Liq_Exhaustion` never fires for XRP. The strategy is effectively dead for any symbol not named BTC. **Root Cause:** Symbol-agnostic absolute threshold instead of relative or per-symbol calibration. **Fix:**

```
# Replace hardcoded threshold with a per-symbol relative floor (e.g. 2x the median of 2
period mean):
liq_combined_mean = (mean_liq_long + mean_liq_short) / 2.0
# Require the candle's liq to be at least 3x the 20-period rolling mean AND
# the 20-period mean to be at least 2x the median to filter out dead-market noise
liq_long_hist_sorted = np.sort(liq_l_arr[-20:])
liq_short_hist_sorted = np.sort(liq_s_arr[-20:])
median_liq_long = liq_long_hist_sorted[len(liq_long_hist_sorted)//2] if
len(liq_long_hist_sorted) > 0 else 0.0
median_liq_short = liq_short_hist_sorted[len(liq_short_hist_sorted)//2] if
len(liq_short_hist_sorted) > 0 else 0.0

# Symbol-aware floor: scaled to median to avoid dead-symbol issues
per_symbol_floor = max(100.0, 5.0 * median_liq_long) # at least $100, scales with
liquidity
liq_l_spike = (mean_liq_long >= per_symbol_floor and
liq_l_val / (mean_liq_long + 1.0) >= 3.0)

per_symbol_floor_s = max(100.0, 5.0 * median_liq_short)
liq_s_spike = (mean_liq_short >= per_symbol_floor_s and
liq_s_val / (mean_liq_short + 1.0) >= 3.0)
```

🐛 **BUG H-6 · OBI staleness window is 45 seconds but DOM scrape interval is shorter** — the 0.15 hard gate causes deterministic signal kills when OBI dips slightly below 0.15

Location: `_evaluate_m1()` lines 1090–1110 **Impact:** When OBI data is *available* (within 45s), the gate at line 1097/1101 is binary: `fire_long=False` if `obi_10 < 0.15`. But `obi_confidence` is computed as `max(0.0, min(1.0, (abs(obi_10) - 0.15) / 0.45))` — meaning at `obi_10 = 0.15` exactly, confidence is 0.0, which means $(0.5 + 0.5 * 0.0) = 0.5\times$ risk in line 2190 — the same penalty as completely stale OBI. So the strategy treats “OBI fresh but neutral” identically to “OBI dead”. Moreover, the hard cutoff at 0.15 means `OBI=0.149` kills the trade entirely while `OBI=0.150` fires it at 50% risk. This is a cliff edge, not a gradient. Worse: when OBI is stale (line 1109), `obi_confidence = 0.5` is assigned but `fire_long/fire_short` are *not* zeroed. So on a stale OBI, the bot fires at half size in *both directions*

whenever any other condition triggers — exactly the opposite of risk-off behavior. **Root Cause:** The "soft gate" actually has a hard floor at 0.15 plus a confusing 50% penalty equivalence. **Fix:**

```
obi_10 = getattr(self, "current_live_obi_10", 0.0)
obi_age = time.time() - getattr(self, "last_obi_time", 0.0)
obi_data_available = obi_age < 45.0

if obi_data_available:
    # Soft gradient: 0.05 threshold = 0% confidence, 0.40 = 100% confidence
    obi_confidence_long = max(0.0, min(1.0, (obi_10 - 0.05) / 0.35))
    obi_confidence_short = max(0.0, min(1.0, (-obi_10 - 0.05) / 0.35))
    for sname, s in new_signals.items():
        if s.get("fire_long"):
            if obi_confidence_long < 0.20: # below 20% confidence → reject
                s["fire_long"] = False
            else:
                s["obi_confidence"] = obi_confidence_long
        if s.get("fire_short"):
            if obi_confidence_short < 0.20:
                s["fire_short"] = False
            else:
                s["obi_confidence"] = obi_confidence_short
    else:
        # Stale OBI = high uncertainty → REJECT entries, do not just penalize size
        _log.warning("[ML] [%s] OBI stale (%.1fs). Suppressing all entries this evaluation cycle.",
                    self.symbol, obi_age)
    for sname, s in new_signals.items():
        s["fire_long"] = False
        s["fire_short"] = False
        s["obi_confidence"] = 0.0
```

🐞 **BUG H-7** · `manage()` `live_pnl` cache is written under `_trade_lock` (line 1727) but `live_pnl()` reads `t.get("live_pnl")` outside the lock (line 2032)

Location: `manage()` line 1727 vs. `live_pnl()` line 1999–2032 **Impact:** `manage()` updates `t["live_pnl"]` on the *local copy* `t` (because `t = self.active_trades[name].copy()` at line 1705) and commits back at line 1988 (`self.active_trades[name].update(t)`). The commit IS under `_trade_lock`. But the copy `t = ...` at line 1705 is also under the lock, so far so good. **However:** between the copy and the commit, the local `t["volume"] = float(pos.volume)` write at line 1726 and `t["live_pnl"] = float(pos.profit)` at line 1727 happen on a local copy that is potentially stale by the time `update()` is called at line 1988. If another `manage()` invocation runs concurrently (different thread), the second invocation's commit may clobber the first's `partial_exits_done` increment. **Root Cause:** Read-modify-write pattern without optimistic concurrency control or versioning. **Fix:** Either serialize `manage()` calls per executor (use a separate `_manage` lock and ensure only one `manage()` thread runs per executor at a time):

```
def __init__(self, symbol):
    ...
    self._manage_lock = threading.Lock()
    self.engine_ref = None

def manage(self, bid, ask):
    if not self._manage_lock.acquire(blocking=False):
        # Already managing – skip this tick to prevent re-entrant clobbering
        return
    try:
        ... # existing body
    finally:
        self._manage_lock.release()
```

🐞 **BUG H-8** · `is_live` window of 300s causes stale Coinglass replays to be misclassified as live, contaminating `all_candles_dict[current_ts_sec]` with replay data

Location: `update()` line 637 **Impact:** `is_live = abs(current_ts_sec - now_sec) < 300`. If Coinglass replays a 4-minute-old candle (common when the WebSocket reconnects), `abs(current_ts_sec - now_sec)` will be 240, which is `< 300` → marked live. The bot then *overwrites* the current 5m candle's OHLC/CVD with the replayed data — corrupting the live candle. Subsequently the boundary detector at line 687 (`current_ts_sec > self.last_ts`) sees a newer `current_ts_sec` and rebuilds queues with corrupted data. **Root Cause:** Window is too loose (300s = the full candle width); should be `≤ 60s` (1 candle's worth of normal replay latency). **Fix:**

```
# Tighten live window to 90s. A live candle's timestamp should never be more than ~90s
the past for a 5m bar.
```

```
is_live = -60 <= (now_sec - current_ts_sec) <= 360 - 60 # i.e. the candle is current
(started up to 5min ago, not in the future)
# Even better: align to true 5m boundary
current_bar_start = (now_sec // 300) * 300
is_live = (current_ts_sec == current_bar_start)
```

🐛 BUG H-9 · `_calculate_rsi_list` uses `gains[:period]` for initial SMA — should use `gains[:period+1]` for proper Wilder's bootstrap (off-by-one in seed RSI)

Location: `_calculate_rsi_list()` lines 310–328 **Impact:** `deltas = np.diff(closes)` has length `len(closes)-1`. The initial RSI value should be at index `period` of `closes`, using `gains[0:period]` (which corresponds to `deltas` between `closes[1:period]` and `closes[0:period-1]`). The current code does this correctly *for the SMA*, but then the Wilder loop iterates `range(period+1, len(closes))` and uses `gains[i-1]` — at the first iteration (`i = period+1`), it uses `gains[period]`. The relationship between `i` and `gains[i-1]` is consistent, but the **first RSI value at index `period` uses `gains[:period]` (indices `0..period-1`)**, then the Wilder update at `i=period+1` uses `gains[period]` (the *next* gain), which is correct. After audit, the indexing is actually right, **BUT** the issue is that for `len(closes) == period+1` (exactly 15 closes for `period=14`), the function returns 14 values of 50.0 and 1 valid value at index `period`. This means the first 14 candles have a meaningless `rsi=50.0` baseline, which then enters the deque and is used for `rsi_arr[-1]` calculations — corrupting `vwap_long = (... rsi < 35 ...)` checks during warmup until 14 candles have rolled through. **Root Cause:** Returning 50.0 placeholders is OK for warmup but no flag/filter prevents downstream usage of placeholder RSI in strategy gating. **Fix:** Add a warmup gate that explicitly checks for the placeholder.

```
def _calculate_rsi_list(self, closes, period=14):
    rsi_list = [None] * len(closes) # Use None instead of 50.0 to mark unset
    if len(closes) <= period:
        return rsi_list
    ...
    return rsi_list

# In seed_from_binance_api, replace placeholder None with 50.0 ONLY in candle dict but mark
warmup
for i, rsi_val in enumerate(rsi_values):
    parsed_candles[i]["rsi"] = rsi_val if rsi_val is not None else 50.0
    parsed_candles[i]["rsi_warmup"] = (rsi_val is None)

# In _evaluate_ml, skip strategies that need RSI if any of the last N candles are in warmup:
rsi_warmup_recent = any(c.get("rsi_warmup", False) for c in sampled_snap[-15:])
if rsi_warmup_recent:
    _log.info("[ML] [%s] RSI still warming up. RSI-dependent strategies suspended.",
self.symbol)
    # Suppress only RSI-dependent strategies, allow CVD_Momentum/Breakout to continue
    rsi_dependent = {"ML_High_PF", "ML_High_Trades", "ML_Ultra_Freq", "VWAP_Bounce",
"Liq_Exhaustion"}
    for sname in list(new_signals.keys()):
        if sname in rsi_dependent:
            new_signals[sname]["fire_long"] = False
            new_signals[sname]["fire_short"] = False
```

🟡 MEDIUM — Logic flaws / silent strategy blockers

🐛 BUG M-1 · `fr_extreme_short` requires funding > +3% which never happens on perpetuals; should be funding-rate-per-period not annualized

Location: `_evaluate_ml()` lines 916–917

```
fr_extreme_long = fr_current < -0.02 and fr_current < fr_20_avg - 0.01
fr_extreme_short = fr_current > 0.03 and fr_current > fr_20_avg + 0.01
```

Impact: Binance perpetual funding rate per 8-hour interval is typically $\pm 0.01\%$ to $\pm 0.1\%$, expressed as decimal (0.0001 to 0.001). Even extreme rates rarely exceed 0.0075 (0.75% per period). The threshold of >0.03 (3%) is essentially unreachable — `fr_extreme_short` will fire on average **once every several years**. `fr_extreme_long` at -0.02 is similarly unreachable. The 1.5x risk boost in `_on_update()` (lines 2183–2188) is therefore dead code. **Root Cause:** Mixing up annualized vs. per-period funding rate, and asymmetric thresholds (-2% vs +3% — no asymmetric justification). **Fix:**

```
# Per-period extremes — calibrated for Binance 8h funding intervals
fr_extreme_long = (fr_current < -0.0010) and fr_current < fr_20_avg - 0.0005
fr_extreme_short = (fr_current > 0.0010) and (fr_current > fr_20_avg + 0.0005)
```

🐛 **BUG M-2** · `vwap_long/vwap_short` use `live_low/live_high` from current incomplete bar, but `rsi/cvd_flip` from completed `last_closed` — **time inconsistency** causes phantom triggers

Location: `_evaluate_ml()` lines 1027–1028

```
vwap_long = (live_low <= vwap_lower and rsi < 35 and cvd_flip > 0)
vwap_short = (live_high >= vwap_upper and rsi > 65 and cvd_flip < 0)
```

Copy

Impact: `live_low` is the running low of the *current* incomplete 5m candle (line 923). `rsi` is from `rsi_arr[-1]` which is the RSI of the *last closed* sampled candle. `cvd_flip` is the velocity between the last two *closed* candles (line 798). The strategy fires when the current candle pokes below VWAP lower but the RSI/CVD are from up to 5 minutes ago. The signal can fire 30 seconds into a new bar based on stale RSI < 35 from the previous bar, while the new bar may already be aggressively recovering. False entries on noise wicks are extremely likely. **Root Cause:** Mixing intra-bar and bar-close indicators without a confirmation requirement. **Fix:**

```
# Require that the LAST CLOSED bar also touched vwap_lower (confirmed)
# AND the live bar is currently at/below vwap_lower for added intra-bar conviction
last_low = last_closed["low"]
last_high = last_closed["high"]
vwap_long = (last_low <= vwap_lower and live_low <= vwap_lower and rsi < 35 and cvd_flip > 0)
vwap_short = (last_high >= vwap_upper and live_high >= vwap_upper and rsi > 65 and cvd_flip < 0)
```

Copy

🐛 **BUG M-3** · `recover_positions()` uses **estimated SL** of `entry * 0.99` when `broker.p.sl == 0` — for a long that's recovered at \$60,000, this places `virtual_sl` at \$59,400 (1% risk) regardless of actual original risk; trailing stop logic then activates at a wildly wrong `lock_price`

Location: `recover_positions()` lines 1192–1194 **Impact:** Recovered positions get arbitrary risk values. `risk = abs(entry - sl)` becomes `entry * 0.01`. Then in `manage()`: `lock_price = entry + risk * rr * lock_pct = entry + entry*0.01 * 3.0 * 0.30 = entry * 1.009`. The trail activates at 0.9% profit instead of the original strategy's intended R-multiple. TP gets `entry * 1.03`, which may already be past the original TP. **Root Cause:** No way to recover original strategy intent from broker order alone; should query `history_orders` or `comments`. **Fix:**

```
# Try to parse the magic-encoded strategy and apply that strategy's CONFIG defaults rather than guessing
sl = float(p.sl) if p.sl > 0 else None
tp = float(p.tp) if p.tp > 0 else None

# Pull s_config defaults
s_config_map = {
    "ML_High_PF": {"rr": 15.0, "use_trail": True, "lock_pct": 0.30, "trail_pct": 0.70},
    "ML_High_Trades": {"rr": 10.0, "use_trail": True, "lock_pct": 0.25, "trail_pct": 0.60},
    "ML_Ultra_Freq": {"rr": 10.0, "use_trail": True, "lock_pct": 0.35, "trail_pct": 0.75},
    "CVD_Momentum": {"rr": 3.0, "use_trail": True, "lock_pct": 0.30, "trail_pct": 0.70},
    "CVD_Breakout": {"rr": 4.0, "use_trail": True, "lock_pct": 0.25, "trail_pct": 0.50},
    "VWAP_Bounce": {"rr": 2.0, "use_trail": False, "lock_pct": 0.0, "trail_pct": 0.0},
    "Liq_Exhaustion": {"rr": 3.0, "use_trail": True, "lock_pct": 0.35, "trail_pct": 0.75},
}

cfg = s_config_map.get(strat_name, {"rr": 3.0, "use_trail": True, "lock_pct": 0.30, "trail_pct": 0.70})

# Don't fabricate SL; if broker has no SL, mark trade as un-managed and warn
if sl is None:
    _log.error("[ML-Executor] [%s] [%s] Recovered ticket %d has no broker SL. Skipping recovery - manual intervention required.",
               self.symbol, strat_name, ticket)
    continue

risk = abs(entry - sl)
if tp is None:
    tp = (entry + risk * cfg["rr"]) if side == "LONG" else (entry - risk * cfg["rr"])

self.active_trades[strat_name] = {
    ..., "rr": cfg["rr"], "use_trail": cfg["use_trail"],
    "lock_pct": cfg["lock_pct"], "trail_pct": cfg["trail_pct"], ...
}
```

Copy

Copy

🐛 **BUG M-4** · `live_pnl()` does not include `commission/swap` for **OPEN** positions, only realized at close — **leverage cap** calculations and dashboard show **inflated nominal PnL**

Location: `live_pnl()` lines 2030–2032 and `manage()` line 1727 **Impact:**

`mt5.symbol_info_tick(...).profit` gives floating profit but typically excludes pending commission and

swap. For high-frequency trades with commissions of \$5-\$15 per turn, the floating PnL can be off by 5-10% per trade. More importantly, the drawdown circuit breaker at line 2126 (`daily_dd >= 75`) uses `session_pnl` which is post-close — but during open trades, large floating losses won't trip the breaker.

Root Cause: Floating PnL only reflects price P&L, not commissions/swaps until realized. **Fix:** Approximate per-trade fee load and subtract from cached PnL:

```
# In manage(), after t["live_pnl"] = float(pos.profit):
# Estimate round-trip commission (taker fees ~0.04% each side = 0.08% total for crypto CFDs)
fee_estimate = entry * t.get("volume", 0.0) * 0.0008 # rough taker round-trip
t["live_pnl"] = float(pos.profit) - fee_estimate
```

And use floating equity in the drawdown breaker:

```
# In _on_update at line ~2124:
floating_pnl = ex.live_pnl()
realized_dd = -ex.session_pnl if hasattr(ex, 'session_pnl') else 0.0
daily_dd = realized_dd + max(0.0, -floating_pnl) # include unrealized losses
```

🐞 **BUG M-5 · `coinglass_entry` priced from Coinglass but broker fill is from MT5 — `risk_val` from signal is in Coinglass price-space, used in MT5 `Entry` order → broker rejects on mismatch or fills with skewed risk**

Location: `open_trade()` lines **1402–1418** **Impact:** If Coinglass shows BTC at \$60,000 and MT5 shows BTCUSD.p at \$60,050 (CFD basis), then `coinglass_entry` = 60000 and `entry` = 60050. The risk is computed as `abs(coinglass_entry - sl_price) = abs(60000 - 59700) = 300`. Then `virtual_sl` = `entry - risk` = 59750. The actual price distance from MT5 ask is `300` = 0.5%, ok. But the SL price (`virtual_sl=59750`) is from MT5 space, while the *signal's* `sl_price=59700` was meant for Coinglass space. The catastrophic SL on the broker (`cat_sl` = `entry - risk * 2.0` = 59450) is in MT5 space — fine. But all the **strategy-internal trailing math** (`highest_peak`, `lock_price`) uses MT5 prices while the original signal expected Coinglass prices. The trail activates at `entry + risk * rr * lock_pct`, which is correct in MT5 space, so trailing works. **HOWEVER**, the `tp_price` returned by `_evaluate_ml` is in Coinglass space, and is passed to MT5 unchanged — TP hits at the wrong price. **Root Cause:** No basis-adjustment between Coinglass and MT5. **Fix:** Always re-anchor signal TP/SL to MT5 entry preserving the original *risk distance and RR*, not the absolute Coinglass prices:

```
# After fetching MT5 entry:
if coinglass_entry and coinglass_entry > 0:
    # Compute signal's risk in price space (basis-invariant)
    sig_risk = abs(coinglass_entry - sl_price)
    sig_rr = abs(tp_price - coinglass_entry) / max(sig_risk, 1e-9)
    # Rebuild SL/TP in MT5 space
    sl_price = (entry - sig_risk) if side == "LONG" else (entry + sig_risk)
    tp_price = (entry + sig_risk * sig_rr) if side == "LONG" else (entry - sig_risk * sig_rr)
    risk_val = sig_risk
```

🐞 **BUG M-6 · `max_hold_hit` of `75 * 300 = 22500s` (6.25 hours) is hardcoded but **VWAP_Bounce** strategies typically resolve in 1-2 bars — holding for 6h on a mean-reversion miss accumulates negative carry & swap**

Location: `manage()` line **1895** **Impact:** All strategies share a 75-candle maximum hold time. For VWAP_Bounce (mean-reversion, `rr=2.0`, no trail), holding past 5–10 bars usually means the bounce thesis is invalidated. Holding to 75 bars guarantees swap fees and increases probability of full SL hit. **Root Cause:** Single hardcoded hold-time across heterogeneous strategies. **Fix:**

```
# Per-strategy max hold times in candles:
MAX_HOLD_BARS = {
    "ML_High_PF": 75, "ML_High_Trades": 50, "ML_Ultra_Freq": 30,
    "CVD_Momentum": 75, "CVD_Breakout": 100,
    "VWAP_Bounce": 10, # mean-reversion: resolve fast
    "Liq_Exhaustion": 20 # liquidation bounces resolve quickly
}
strat_maxBars = MAX_HOLD_BARS.get(name, 75)
max_hold_hit = time_elapsed >= strat_maxBars * 300
```

■ LOW — Resiliency / hygiene

🐞 **BUG L-1 · `seed_from_binance_api()` HTTP failure has no retry; if Binance API is rate-limited (429) at startup, bot starts with zero Binance can**

Location: `seed_from_binance_api()` lines **220, 294–297** **Impact:** A single HTTP failure on startup

means the bot relies only on G-Drive/local cache. If neither is present, it cold-starts and `_rebuild_queues()` self-healing re-attempts but inside a lock path (see H-3). **Root Cause:** No retry/backoff on transient HTTP errors. **Fix:**

```
def seed_from_binance_api(self, retries=3, backoff=2.0):  
    import requests  
    last_err = None  
    for attempt in range(retries):  
        try:  
            res = requests.get(url, params=params, timeout=10)  
            if res.status_code == 200:  
                # ... process and return  
                return  
            elif res.status_code == 429:  
                _log.warning("[ML] [%s] Binance rate-limited (429), retry %d/%d after  
%.1fs",  
                            self.symbol, attempt+1, retries, backoff * (2**attempt))  
                time.sleep(backoff * (2**attempt))  
                continue  
            else:  
                last_err = f"HTTP {res.status_code}: {res.text[:200]}"  
        except Exception as e:  
            last_err = str(e)  
            time.sleep(backoff * (2**attempt))  
    _log.error("[ML] [%s] Binance seed failed after %d retries: %s", self.symbol, retries,  
              last_err)
```

🐛 **BUG L-2** · `play_sound()` and `_fire_with_deferred_cooldown()` are launched as daemon threads without exception capture — silent failures hide trade execution errors

Location: `_on_update()` lines 2233, 2236–2237, 2245, 2248 **Impact:**

`threading.Thread(target=_fire_with_deferred_cooldown, args=(...), daemon=True).start()` swallows ANY exception (uncaught `KeyError`, `TypeError`, MT5 segfault). The pre-reserved `pending_trades` slot is never released because the failure path is in the `else: success=False` branch only if `open_trade()` returned normally. If `open_trade()` raises, the slot leaks forever. **Root Cause:** No try/except wrapper around the threaded callable; thread exceptions go to `stderr` but don't unwind `pending_trades`. **Fix:**

```
def _fire_with_deferred_cooldown(side, strat_name, sig, cool_key, scaled_risk_usd):  
    try:  
        success = ex.open_trade(side, strat_name, sig["sl"], sig["tp"], sig["risk"],  
                                f"{'[DRV] ' if DRV_RUN_MODE else ''}{strat_name} {side} (SL={sig['sl']:.2f}, TP=  
{sig['tp']:.2f})",  
                                sig.get("coinglass_entry"), scaled_risk_usd=scaled_risk_usd)  
        if success:  
            with self.lock:  
                self.signal_cool[cool_key] = time.time()  
        else:  
            with ex_trade_lock:  
                ex.pending_trades.discard(strat_name)  
    except Exception as e:  
        _log.exception("[ML-Executor] [%s] [%s] open_trade thread crashed: %s", symbol,  
                      strat_name, e)  
        with ex_trade_lock:  
            ex.pending_trades.discard(strat_name)
```

🐛 **BUG L-3** · `save_candles_to_cache()` spawns a daemon thread on every candle boundary (line 691) but never joins — under sustained high-frequency boundaries (multiple symbols), threads pile up

Location: `update()` line 691 **Impact:** Each 5-minute close on each symbol spawns a daemon thread.

Threads complete quickly (~50ms for CSV write), so leak is small. But on cold start, `_rebuild_queues()` from `seed_from_*` triggers `save_candles_to_cache()` synchronously (line 205), which is fine. The issue is the **CSV writer is not lock-protected against itself** — two threads writing the same `seed_data_BTCUSDT.csv` simultaneously corrupts the file. **Root Cause:** No file-level lock around `df.to_csv()`. **Fix:**

```
# Add a per-symbol file lock at class level  
def __init__(self, symbol):  
    ...  
    self._cache_lock = threading.Lock()  
    ...  
  
def save_candles_to_cache(self):  
    if not self._cache_lock.acquire(blocking=False):
```

```

        return # already saving; skip
    try:
        # ... existing body ...
    finally:
        self._cache_lock.release()

```

Summary Triage Matrix

Severity	Bug ID	Area	Effect
● CRITICAL	C-1	manage / partial close	Volume desync → over-close → unbounded loss
● CRITICAL	C-2	open_trade / VWAP TP	TP silently flips through entry → instant fill-loss
● CRITICAL	C-3	7-cap is per-symbol	Real cap is 7×N_symbols → over-exposure
● CRITICAL	C-4	CVD_Momentum short	Fires on bullish OI absorption → whipsaw losses
● CRITICAL	C-5	CVD baseline persist	Stale offset corrupts CVD-dependent gates
■ HIGH	H-1	_rebuild_queues mutation	G-Drive CVD overwritten → strategy gates wrong
■ HIGH	H-2	candle_lock ↓	Torn reads during rebuild
■ HIGH	H-3	seed during rebuild	Lock deadlock + unprotected dict writes
■ HIGH	H-4	_evaluate_ml deque race	Concurrent np.array build → ValueError
■ HIGH	H-5	Liq_Exhaustion \$10K floor	Strategy dead for XRP/non-BTC symbols
■ HIGH	H-6	OBI gradient cliff	Stale OBI fires at half size in both dirs
■ HIGH	H-7	manage() RMW race	Re-entrant manage clobbers partial_exits_done
■ HIGH	H-8	is_live window 300s	Stale replays corrupt live candle
■ HIGH	H-9	RSI placeholder 50	RSI-dependent strats fire on placeholder during warmup
■ MEDIUM	M-1	fr_extreme thresholds	1.5x boost is dead code
■ MEDIUM	M-2	VWAP intra-bar mixing	Phantom triggers on wick noise
■ MEDIUM	M-3	recover_positions fab SL	Wrong trail/lock on recovered trades
■ MEDIUM	M-4	live_pnl ignores fees	DD breaker under-trips
■ MEDIUM	M-5	Coinglass↔ ↓ basis	TP/SL skewed by basis offset
■ MEDIUM	M-6	75-bar hold all-strats	VWAP_Bounce holds 6h on miss
■ LOW	L-1	Binance HTTP no-retry	Cold start risk
■ LOW	L-2	Thread exceptions swallowed	Pending slot leak
■ LOW	L-3	CSV concurrent write	File corruption

Recommended Patch Order

1. C-3 (per-symbol cap leak) — *before next live deploy*; structurally limits blast radius
2. C-5 + H-1 + H-2 + H-3 + H-4 (the CVD/lock cluster) — these compound and corrupt every strategy
3. C-1 + C-2 (execution bugs that silently lose money)
4. C-4 + H-5 + H-6 + H-9 (strategy logic correctness)
5. H-7 + H-8 + M-2 + M-5 (data-time consistency)
6. M-1, M-3, M-4, M-6 (calibration)
7. L-1, L-2, L-3 (hygiene)

I recommend **freezing live execution and forcing DRY_RUN=true** until at least the 5 CRITICAL and top-tier HIGH items (H-1/H-2/H-3/H-4) are patched, since the CVD pipeline corruption (H-1) silently invalidates gating used by 4 of 7 strategies. The DRY-RUN partial-close codepath in C-1 masks the bug entirely, so

paper-trading results since the v5.0 fix-cycle should be treated as **unreliable** for sizing-related validation.



Would you like me to:

1. Generate a **single unified patch file** (.diff) you can apply directly with `git apply`, or
2. Produce a **regression test harness** (pytest) that locks down the fixed behavior for the CVD-pipeline / partial-close paths, or
3. Re-audit a **specific subsystem** (e.g., MT5 order routing or the Coinglass DOM scraper integration in the parent `live_orderflow_bot`)?



Generate a unified diff patch for all the critical fixes you identified in `'live_ml_bot.py'`, grouped by severity and ready to apply. →

Create a pytest regression test harness that reproduces and locks down the fixed behavior for the CVD pipeline, partial-close logic, and thread-safety paths. →

Re-audit the MT5 order routing subsystem only and produce a focused report on race conditions, duplicate orders, partial fills, and shutdown safety. →

Turn your findings into a prioritized remediation checklist with estimated implementation effort, blast radius, and verification steps for each bug. →

